



Data Structure

(Course Code: 25B11CI318)

Red Black Trees

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

Waknaghat, Solan, HP-173234

Why Red-Black Trees?

An ordinary BST may have height:

$$h = n - 1.$$

Its search, insertion and deletion can therefore take:

$$O(n).$$

Red-Black Tree

A red-black tree uses color constraints to maintain logarithmic height.

Search, insertion and deletion = $O(\log n)$ worst case

What is a Red-Black Tree?

Definition

A red-black tree is a binary search tree in which each node has a color, red or black, and the coloring satisfies additional properties that bound the tree's height.

- Keys obey the BST ordering property.
- Color controls structural balance.
- Updates use recoloring and rotations.
- Missing children are represented by black NIL leaves.

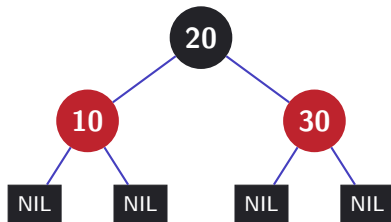
The Five Red-Black Properties

- ① Every node is red or black.
- ② The root is black.
- ③ Every NIL leaf is black.
- ④ If a node is red, both its children are black.
- ⑤ From each node, every path to a descendant NIL leaf contains the same number of black nodes.

Consequence

Two red nodes cannot be adjacent on a parent-child path. Two black nodes may be adjacent.

Example with Explicit NIL Leaves



- Root 20 is black.
- Red nodes 10 and 30 have black NIL children.
- Every root-to-NIL path has the same black count.

NIL Leaves and Stored Nodes

- Stored nodes contain keys.
- NIL leaves represent missing children.
- A stored node with no key-bearing children still has two conceptual NIL children.
- Later diagrams may omit NIL leaves for readability.

Convention

Every omitted child is a black NIL leaf. The number n counts stored, key-bearing nodes only.

Black-Height

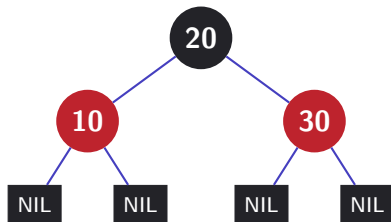
Definition

The black-height $bh(x)$ is the number of black nodes on any path from, but not including, x to a descendant NIL leaf.

- Exclude x itself.
- Include the terminal NIL leaf.
- The red-black properties make this count identical across all such paths.

$$bh(NIL) = 0$$

Black-Height Example



$$bh(10) = bh(30) = 1.$$

From 20, each path contains one black node after 20: the terminal NIL.

$$bh(20) = 1.$$

Two Common Violations

Red-Red Violation

A red node has a red parent.
This commonly arises when inserting a new red node.

Unequal Black Counts

Some paths contain fewer black nodes than others.
This can arise when deleting a black node.

Repair Operations

Recoloring changes black counts. Rotations change local structure while preserving BST order.

Why the Height is Logarithmic

A subtree rooted at x contains at least:

$$2^{bh(x)} - 1$$

stored nodes.

Thus, for the root:

$$n \geq 2^{bh(root)} - 1.$$

Therefore:

$$bh(root) \leq \log_2(n + 1).$$

Because consecutive red nodes are forbidden, a path has at most about twice as many nodes as its black count:

$$h \leq 2 \log_2(n + 1).$$

Here h is the edge height to the deepest stored node.

Why the Subtree-Size Bound Holds

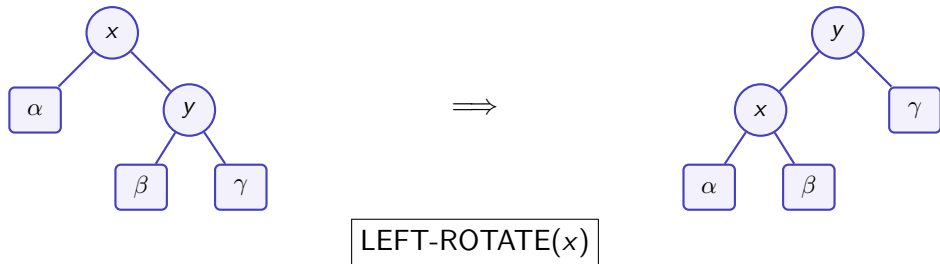
Use induction on subtree height.

- A NIL subtree contains 0 stored nodes.
- Each child of x has black-height at least $bh(x) - 1$.
- Each child subtree therefore contains at least $2^{bh(x)-1} - 1$ stored nodes.

Including x :

$$1 + 2 \left(2^{bh(x)-1} - 1 \right) = \boxed{2^{bh(x)} - 1}.$$

Rotation: The Same BST Operation



y moves up, x moves down, and β becomes the right subtree of x .

Rotations Preserve Inorder Ordering

Before and after the rotation:

$$\alpha < x < \beta < y < \gamma.$$

Hence the inorder sequence remains:

$$\alpha, x, \beta, y, \gamma.$$

- A rotation takes $O(1)$ time.
- RIGHT-ROTATE is the inverse structural operation.
- A rotation alone does not automatically restore all color properties.

Red-Black Repair

The algorithm chooses rotations and recoloring together.

Implementation Conventions

The algorithms use:

- $T.root$: pointer to the root.
- $T.nil$: shared black sentinel.
- Node fields: key, color, left, right and parent.
- Distinct keys, with duplicate insertion rejected.

Sentinel Initialization

The sentinel is black. Its left and right pointers refer to itself. An empty tree has $T.root = T.nil$.

Deletion may temporarily update the sentinel's parent to identify the position requiring repair.

Node Representation in C

```
1 typedef enum {  
2     RED,  
3     BLACK  
4 } Color;  
5  
6 typedef struct Node {  
7     int key;  
8     Color color;  
9     struct Node *left;  
10    struct Node *right;  
11    struct Node *parent;  
12 } Node;  
13  
14 typedef struct {  
15     Node *root;  
16     Node *nil;  
17 } RBTtree;
```

Red-Black Insertion

- 1 Locate the insertion position using BST search.
- 2 Create the new node z .
- 3 Set both children to the black sentinel.
- 4 Color z red.
- 5 Repair any red-red violation.
- 6 Ensure the root is black.

Why Insert Red?

Replacing a NIL position with a red node and two NIL children preserves the black count along that path.

When Insertion Needs Repair

- If the new node is the root, color it black.
- If its parent is black, no red-red violation exists.
- If its parent is red, examine the uncle.

Let:

z = current node,

$p = z.parent$, $g = p.parent$,

u = the other child of g .

The repair case depends on the uncle's color and the orientation of z and p .

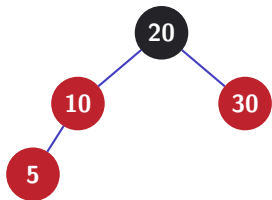
Insertion Cases

Assume the parent is the left child of the grandparent.

Case	Condition	Action
1	Uncle is red	Recolor parent and uncle black, grandparent red. Continue upward.
2	Uncle is black, z is a right child	Rotate left at the parent to obtain Case 3.
3	Uncle is black, z is a left child	Recolor parent and grandparent, then rotate right at the grandparent.

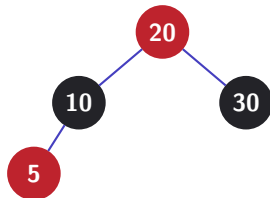
When the parent is a right child, interchange left and right. A NIL uncle is black.

Insertion Case 1: Red Uncle



Before.

Set z to the grandparent and continue upward. If 20 is the root, color it black at the end.



After recoloring.

Why Case 1 Recoloring Works

Before repair:

- The grandparent is black.
- The parent and uncle are red.

After repair:

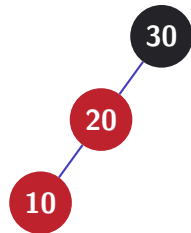
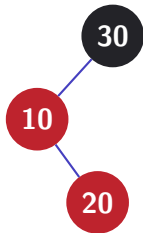
- The parent and uncle become black.
- The grandparent becomes red.

Black Counts

Each path gains a black node at the parent or uncle and loses one at the grandparent. The black count below the grandparent's old position is unchanged.

Insertion Case 2: Triangle

Insert into the right subtree of a red left child.

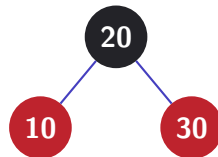
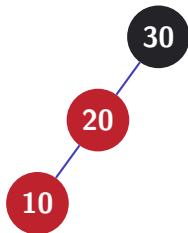


Before.

The triangle becomes a line. Continue with Case 3.

After LEFT-ROTATE(10).

Insertion Case 3: Line



After repair.

Before.

Color 20 black and 30 red, then RIGHT-ROTATE(30).

Insertion: Mirrored Cases

If the parent is the grandparent's right child:

- Red uncle: use the same recoloring rule.
- Triangle: rotate right at the parent.
- Line: recolor and rotate left at the grandparent.

Rotation Bound

Insertion performs at most two primitive rotations. Recoloring may propagate through $O(\log n)$ ancestors.

$$\text{Insertion time} = O(\log n)$$

Insertion Fix-Up: Parent on the Left

```
1 while z.parent.color == RED
2     if z.parent == z.parent.parent.left
3         u = z.parent.parent.right
4
5         if u.color == RED
6             z.parent.color = BLACK
7             u.color = BLACK
8             z.parent.parent.color = RED
9             z = z.parent.parent
10    else
11        if z == z.parent.right
12            z = z.parent
13            LEFT-ROTATE(T, z)
14
15        z.parent.color = BLACK
16        z.parent.parent.color = RED
17        RIGHT-ROTATE(T, z.parent.parent)
```

Insertion Fix-Up: Parent on the Right

Continue the outer conditional:

```
1  else
2      u = z.parent.parent.left
3
4      if u.color == RED
5          z.parent.color = BLACK
6          u.color = BLACK
7          z.parent.parent.color = RED
8          z = z.parent.parent
9      else
10         if z == z.parent.left
11             z = z.parent
12             RIGHT-ROTATE(T, z)
13
14         z.parent.color = BLACK
15         z.parent.parent.color = RED
16         LEFT-ROTATE(T, z.parent.parent)
17
```

Worked Insertion Sequence

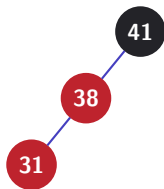
Insert:

41, 38, 31, 12, 19, 8.

Key	Repair
41	First node becomes a black root.
38	Red node under black parent. No repair.
31	Black uncle, line. Recolor and rotate right at 41.
12	Red uncle. Recolor and finish with a black root.
19	Black uncle, triangle. Rotate left at 12, then right at 31 with recoloring.
8	Red uncle. Recolor 12, 31 and 19.

After Inserting 41, 38 and 31

Before repair:



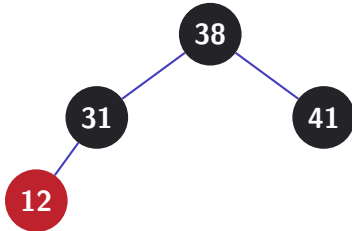
After RIGHT-ROTATE(41) and recoloring:

Root 38_B , left 31_R , right 41_R .
--

After Inserting 12

The uncle of 12 is red node 41.

Recolor 31 and 41 black. The root finishes black.



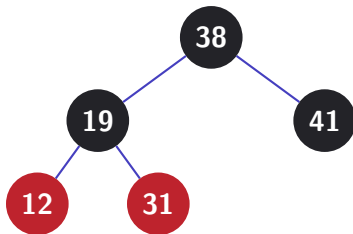
After Inserting 19

Node 19 forms a triangle with 12 and 31.

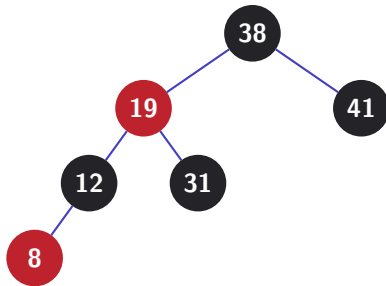
Apply:

LEFT-ROTATE(12), RIGHT-ROTATE(31),

with Case 3 recoloring.



After Inserting 8: Final Tree



Inorder: 8, 12, 19, 31, 38, 41

$$bh(38) = 2$$

Searching in a Red-Black Tree

Search follows ordinary BST rules.

- Smaller key: move left.
- Larger key: move right.
- Equal key: return the node.
- Reach NIL: the key is absent.

Colors

Search does not inspect colors. Colors guarantee that the search path is short.

$$\text{Worst-case search} = O(\log n)$$

Red-Black Deletion

First perform the structural BST deletion.

Let:

- z : the requested target.
- y : the node physically removed.
- x : the child replacing y , possibly NIL.

Color That Matters

Save the original color of y . If y was black, its removal may reduce a path's black count.

If the target has two children, its successor can be used to reduce removal to the zero-child or one-child case.

A Clear Deletion Implementation

For teaching, use key-copying deletion:

- ① If z has at most one non-NIL child, set $y = z$.
- ② Otherwise, let y be the successor of z .
- ③ Copy y 's key and associated item into z .
- ④ Remove y from its original position.
- ⑤ Repair based on y 's original color.

Color Handling

Copy the key and item, not the successor's color. This implementation differs from node-transplant deletion.

Easy Deletion Cases

A Red Node is Removed

No black count changes. No deletion fix-up is needed.

A Black Node is Replaced by a Red Child

Color the replacement child black. The missing black contribution is restored.

A Black Node is Replaced by a Black Position

The replacement may be NIL. A black-count deficit requires further repair.

The Double-Black Interpretation

Meaning

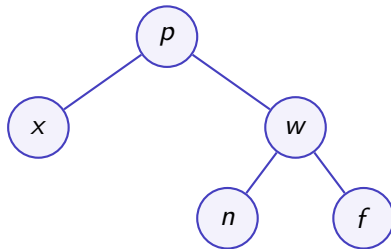
A double-black position represents a subtree that needs one additional black contribution to match other paths.

- It is a reasoning device.
- It is not a third stored node color.
- The affected node x may be the NIL sentinel.
- Fix-up moves or removes the deficit.

If the deficit reaches the root, it can be discarded, because every root-to-leaf path is affected equally.

Deletion Fix-Up Terminology

Assume x is the left child of its parent p .



w : sibling of x .

$n = w.left$: near child.

$f = w.right$: far child.

The Four Deletion Cases

For x as a left child:

Case	Condition	Purpose
1	Sibling red	Obtain a black sibling
2	Sibling black, both children black	Move the deficit upward
3	Sibling black, near child red, far child black	Obtain Case 4
4	Sibling black, far child red	Remove the deficit

For x as a right child, interchange left and right.

Deletion Case 1: Red Sibling

Assume x is a left child and its sibling w is red.

- ① Color w black.
- ② Color parent p red.
- ③ Rotate left at p .
- ④ Recompute $w = x.parent.right$.

Effect

The deficit remains at x , but its new sibling is black. Continue with Case 2, 3 or 4.

A red sibling implies the parent was black before this case.

Deletion Case 2: Black Sibling and Children

Assume w , $w.left$ and $w.right$ are black.

- 1 Color w red.
- 2 Move the deficit upward by setting $x = p$.

What Happens Next?

If the new x is red, color it black and finish.

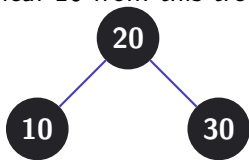
If it is black, continue upward.

If it is the root, discard the extra-black interpretation.

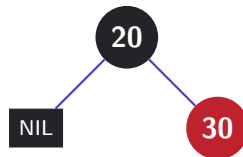
Case 2 performs no rotation.

Case 2: Simple Example

Delete black leaf 10 from this tree:



Before deletion.



After recoloring and finishing at the root.

Deletion Case 3: Near Red, Far Black

For x as a left child:

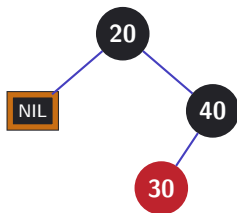
$w.color = BLACK,$ $w.left.color = RED,$ $w.right.color = BLACK.$

- ① Color $w.left$ black.
- ② Color w red.
- ③ Rotate right at w .
- ④ Recompute the sibling.

Effect

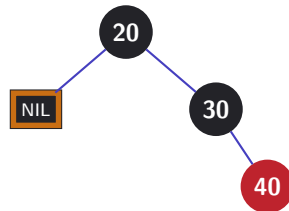
The new sibling has a red far child. Proceed immediately to Case 4.

Case 3: Rotation Diagram



Before Case 3.

Orange double border marks the position carrying the deficit.



After RIGHT-ROTATE(40).

Deletion Case 4: Far Child Red

For x as a left child, sibling w is black and $w.right$ is red.

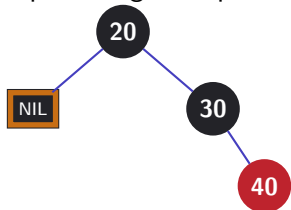
- ① Give w the old color of parent p .
- ② Color p black.
- ③ Color $w.right$ black.
- ④ Rotate left at p .
- ⑤ Finish the repair.

Effect

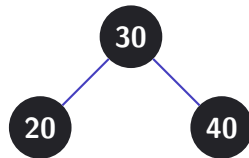
The rotation and recoloring restore equal black counts. The extra-black interpretation is no longer needed.

Case 4: Final Tree

Continue the preceding example.



Before Case 4.



After LEFT-ROTATE(20).

Deletion Example: Remove a Red Leaf

Start from the insertion example:

$38_B, 19_R, 41_B, 12_B, 31_B, 8_R.$

Node 8 is a red leaf.

- Remove 8.
- Replace its position with NIL.
- No black count changes.
- No fix-up is required.

Deleting a red leaf requires no recoloring.

Deletion Example: Then Remove 12

After removing 8, node 12 is a black leaf.

Its sibling is black node 31 with two black NIL children.

- 1 Remove 12.
- 2 Apply Case 2: color 31 red.
- 3 Move the deficit to parent 19.
- 4 Since 19 is red, color it black and finish.

Final tree:

38_B with left 19_B and right 41_B

where 19 has right child 31_R.

Deletion Algorithm: Structural Removal

```
1 RB-DELETE(T, z)
2   y = z
3
4   if z.left != T.nil and z.right != T.nil
5       y = MINIMUM(z.right)
6       copy y's key and associated item into z
7
8   oldColor = y.color
9
10  if y.left != T.nil
11      x = y.left
12  else
13      x = y.right
14
15  replace y by x in y's parent or in T.root
16  x.parent = y.parent    // also when x is T.nil
17
18  if oldColor == BLACK
```

Delete Fix-Up: Cases 1 and 2

For the left-child branch:

```
1 while x != T.root and x.color == BLACK
2     if x == x.parent.left
3         w = x.parent.right
4
5         if w.color == RED                // Case 1
6             w.color = BLACK
7             x.parent.color = RED
8             LEFT-ROTATE(T, x.parent)
9             w = x.parent.right
10
11        if w.left.color == BLACK and
12            w.right.color == BLACK        // Case 2
13            w.color = RED
14            x = x.parent
15    else
16        // Continue with Cases 3 and 4.
```

Delete Fix-Up: Cases 3 and 4

Continue inside the preceding else:

```
1         if w.right.color == BLACK      // Case 3
2             w.left.color = BLACK
3             w.color = RED
4             RIGHT-ROTATE(T, w)
5             w = x.parent.right
6
7         w.color = x.parent.color        // Case 4
8         x.parent.color = BLACK
9         w.right.color = BLACK
10        LEFT-ROTATE(T, x.parent)
11        x = T.root
```

The right-child branch uses the mirrored rules on the following slide. After the loop, set `x.color = BLACK`.

Delete Fix-Up: Mirrored Rules

When x is a right child, set:

$$w = x.parent.left.$$

Case	Mirrored Action
------	-----------------

- | | |
|---|---|
| 1 | Red sibling: recolor sibling black and parent red, then rotate right at the parent. |
| 2 | Both sibling children black: recolor sibling red, then move x to its parent. |
| 3 | Far child $w.left$ black: color $w.right$ black and w red, then rotate left at w . |
| 4 | Give w the parent's color, color parent and $w.left$ black, then rotate right at the parent and finish. |
-

Deletion Rotation Bound

- Case 1 uses one rotation and changes the sibling color.
- Case 2 uses no rotation and may repeat upward.
- Case 3 uses one rotation and immediately leads to Case 4.
- Case 4 uses one rotation and finishes.

CLRS Fix-Up Bound

A deletion uses at most three primitive rotations. Recoloring and upward movement may take $O(\log n)$ time.

$$\text{Deletion time} = O(\log n)$$

Sentinel Details During Deletion

- The replacement x may equal $T.nil$.
- Fix-up still needs $x.parent$.
- Therefore structural deletion assigns the replacement's parent even when the replacement is NIL.
- In a valid fix-up state with an extra-black nonroot, the sibling is not NIL.

Shared Sentinel

A shared sentinel's parent is temporary context for the current deletion. It is not a permanent parent for every missing child in the tree.

Red-Black Tree Complexity

Operation	Worst-Case Time
Search	$O(\log n)$
Insertion	$O(\log n)$
Deletion	$O(\log n)$
Minimum / maximum	$O(\log n)$
Successor / predecessor	$O(\log n)$
One rotation	$O(1)$
Full traversal	$\Theta(n)$

Node storage is $\Theta(n)$. Iterative fix-up with parent pointers uses $O(1)$ auxiliary space.

AVL Trees and Red-Black Trees

Property	AVL	Red-Black
Balance rule	Subtree heights differ by at most 1	Color and black-height properties
Metadata	Height or balance factor	Color
Worst-case height	$O(\log n)$	$O(\log n)$
Insertion rotations	At most 2	At most 2
Deletion rotations	$O(\log n)$	At most 3 in CLRS fix-up
Search and updates	$O(\log n)$	$O(\log n)$

Applications

- Ordered sets and maps.
- Dynamic dictionaries.
- Predecessor and successor queries.
- Range queries on ordered keys.
- Augmented search trees.

Examples of Augmentation

Subtree sizes support rank and selection.

Additional interval information supports interval-search trees.

Worked Problem: Height Bound

Question

Give the standard red-black height upper bound for a tree with 31 stored nodes.

$$h \leq 2 \log_2(n + 1).$$

$$h \leq 2 \log_2(32) = 2(5).$$

$$h \leq 10.$$

Interpretation

This is a general upper bound. It does not assert that height 10 is achievable with 31 nodes.

Worked Problem: Black-Height

Question

A subtree has black-height 3. What is the guaranteed minimum number of stored nodes?

$$n_x \geq 2^{bh(x)} - 1.$$

$$n_x \geq 2^3 - 1.$$

$$\boxed{n_x \geq 7}.$$

Common Mistakes

- Forgetting that NIL leaves are black.
- Counting the starting node in CLRS black-height.
- Assuming two adjacent black nodes are forbidden.
- Treating double black as a third stored color.
- Rotating without the required recoloring.
- Choosing deletion repair from the target's color instead of the physically removed node's original color.
- Forgetting mirrored insertion and deletion cases.

Practice Questions

- 1 Construct a red-black tree by inserting:

10, 20, 30, 15.

- 2 State the insertion repair after adding 30.
- 3 State the repair after adding 15.
- 4 Find the final root's black-height.
- 5 Delete 15, then delete 10.
- 6 What is the maximum number of rotations in one CLRS deletion?

Practice Answer: Insertion

After inserting 10, 20, 30:

- A mirrored line case occurs at 10.
- Recolor and rotate left at 10.

After inserting 15:

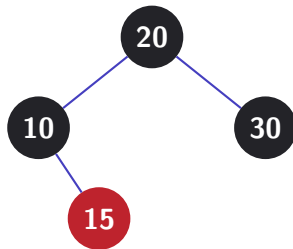
- Its parent 10 and uncle 30 are red.
- Recolor 10 and 30 black.
- Ensure root 20 is black.

Final tree:

20_B with left 10_B and right 30_B

with 15_R as the right child of 10.

Practice Answer: Final Diagram



$$bh(20) = 2.$$

Each path below 20 includes one black stored node and the terminal black NIL.

Practice Answer: Deletion

Delete 15:

- It is a red leaf.
- No deletion fix-up is required.

Then delete 10:

- It is now a black leaf.
- Sibling 30 and its children are black.
- Apply Case 2: color 30 red and move to the root.

Final tree:

20_B with right child 30_R .

One CLRS deletion performs at most $\boxed{3}$ rotations.

Quick Revision

Concept	Key Point
Root	Black
NIL leaves	Black
Red parent	Must have black children
Black-height	Equal black counts to descendant NIL leaves
New node	Initially red
Red uncle during insertion	Recolor and continue upward
Black uncle during insertion	Rotation cases
Black removal	May require deletion fix-up
Height	$O(\log n)$
Search and updates	$O(\log n)$ worst case

Reference

Cormen / CLRS

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein.

Introduction to Algorithms, 3rd edition, MIT Press, 2009.

- Chapter 13: Red-Black Trees.
- Section 13.1: Properties of red-black trees.
- Section 13.2: Rotations.
- Section 13.3: Insertion.
- Section 13.4: Deletion.

The explanations are teaching adaptations. The structural deletion shown uses key copying.

Thank You